



IT 401W - WEB PROGRAMMING AND FRAMEWORKS ACTIVITY NO. 3 - SYSTEM E			SCORE
Clinic Queue Consultation and Billing System			
STUDENT NO.	LAST NAME	FIRST NAME	MIDDLE INITIAL
2023100224	ALCAÑICES	Jhon Emmanuele	S.
STUDENT NO.	LAST NAME	FIRST NAME	MIDDLE INITIAL
2023103275	FULO	Jameo Reign	C.
STUDENT NO.	LAST NAME	FIRST NAME	MIDDLE INITIAL
2023103975	MAHUSAY	Isaac John	P.
COURSE, YEAR, SECTION, AND SET		DATE OF SUBMISSION (mm/dd/yyyy)	INSTRUCTOR
BSIT 4F-G1 SET E		09-30-26	DR. AARON PAUL M. DELA ROSA

System Overview

Purpose

The Clinic Queue Consultation and Billing System is a console-based application designed to manage daily clinic operations. To systematically manage patient types, doctors, priority queues, consultations, bills, discounts, and clinic-operation reports using a robust object-oriented architecture.

Users

Clinic administrative staff, receptionists, and clinic managers

Major Operations

Operation	Description
Register Patient	Registers a new patient into the system.
Register Doctor	Registers a new patient into the system.



Add to Queue	Adds a registered patient into the priority queue.
Call Next Patient	Retrieves the next patient based on polymorphic priority.
Complete Consultation	Completes the consultation and exactly once updates its status, doctor count, and generates a bill.
Search Patient	Searches for an existing patient record using their ID.
Print Bill	Retrieves and prints a specific billing record.
Generate Reports	Generates dynamic reports for waiting patients, completed consultations, financials, and doctor workload.

Business Rules

1. The system strictly rejects duplicate patient IDs, duplicate doctor IDs, and duplicate active queue entries.
2. The queue selects the next patient polymorphically by priority, followed by their queue number.
3. Consultations are rejected if no doctor is available or if data is invalid.
4. Completing a consultation updates its status, doctor count, and bill exactly once.
5. Any failed consultation or billing operation safely leaves all related objects unchanged to maintain state integrity.



Class-Responsibility Table

Class	Role	Responsibilities
Patient	Abstract base class	Stores private patient ID and contact data; defines abstract <code>get_priority</code> and <code>calculate_discount</code> methods.
RegularPatient	Patient subclass	Inherits from Patient. Implements normal priority and standard billing rules. Encapsulates the specific - <code>patient_type: str = "Regular"</code> attribute.
SeniorPatient	Patient subclass	Inherits from Patient. Implements senior priority and specific discount rules. Encapsulates - <code>senior_id: str</code> and - <code>discount_rate: float = 0.20</code> attributes.
EmergencyPatient	Patient subclass	Inherits from Patient. Implements emergency priority and specialized billing rules. Encapsulates the - <code>severity_level: int</code> attribute.
Doctor	Domain class	Manages doctor ID, name, specialization, availability status, and consultation count.
Consultation	Relationship class	Links patient and doctor; tracks queue number, service, charges, status, and completion details.
BillingRecord	Transaction class	Processes the associated consultation, subtotal, discount, final amount, and payment status.
ClinicManager	Manager class	Coordinates patients, doctors, queue logic, consultations, and billing records. Directly responsible for required menu actions including patient searches (<code>search_patient</code>), printing bills (<code>print_bill</code>), and generating formatted reports.



ClinicException	Custom exception	Handles duplicate, queue, doctor, consultation, and billing errors.
------------------------	------------------	---

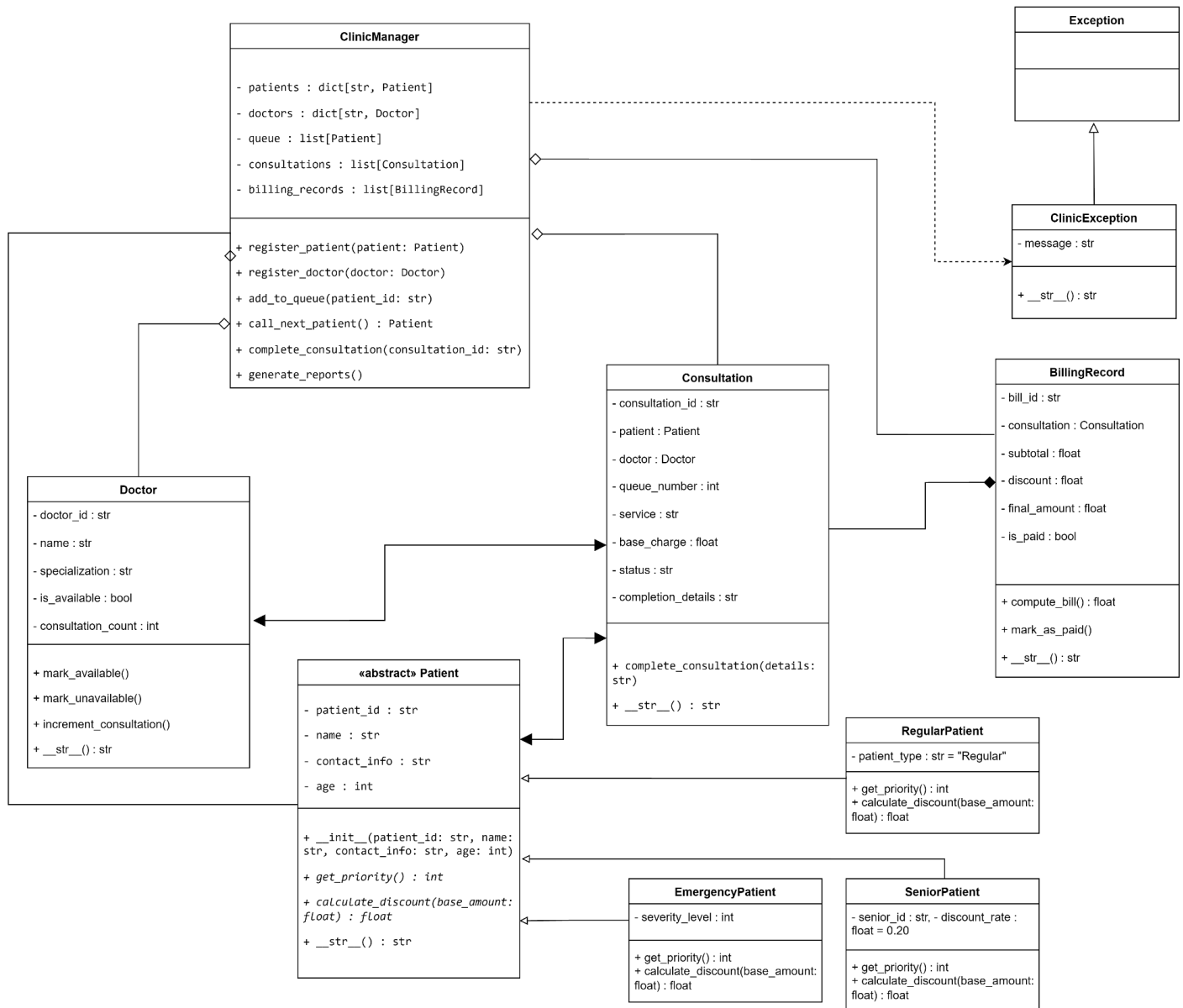
Class Diagram Structure

The following class diagram below represents the object-oriented structure of the Clinic Queue Consultation and Billing System. The diagram below shows the relationships between the objects, including inheritance, composition, aggregation, association, and dependency. It also includes the main attributes (with their visibility) and the methods (with their visibility) for each class.

- **Inheritance and Abstraction** – the abstract Patient class defines the basic set of attributes (patient ID and contact information) that have to be implemented in all subclasses. In this case, all three classes (RegularPatient , SeniorPatient , and EmergencyPatient) introduce their own implementation of the methods `get_priority()` and `calculate_discount()` . The custom exception class ClinicException inherits from the standard Exception class.
- **Composition, Aggregation, and Association** – the ClinicManager class uses aggregation (empty diamonds) to hold references to all patients, doctors, consultations, and billing records, as these objects are not strictly dependent on the manager. The Consultation class is associated with Doctor and Patient , as it only stores information about them. BillingRecord utilizes a strong composition (filled diamond) with Consultation , meaning that a billing record is impossible without a consultation.
- **Encapsulation** – all private and protected attributes and methods are marked with `-` and `~` symbols, respectively. Patient ID, consultation status, and calculated charges are private and should only be accessed via public methods (marked with `+`).



Republic of the Philippines
City of Malolos, Bulacan
BULACAN STATE UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY





FULL RELATIONSHIP LIST

Relationship	Arrow	Code / Diagram Evidence
RegularPatient -> Patient	Hollow triangle	class RegularPatient(Patient):
SeniorPatient -> Patient	Hollow triangle	class SeniorPatient(Patient):
EmergencyPatient -> Patient	Hollow triangle	class EmergencyPatient(Patient):
ClinicException -> Exception	Hollow triangle	class ClinicException(Exception):
Consultation -> Patient	Open Arrow / Solid line	self._patient = patient
Consultation -> Doctor	Open Arrow / Solid line	self._doctor = doctor
BillingRecord -> Consultation	Filled diamond	self._consultation = consultation
ClinicManager -> Patient	Hollow diamond	self.patients: dict[str, Patient]
ClinicManager -> Doctor	Hollow diamond	self.doctors: dict[str, Doctor]
ClinicManager -> Consultation	Hollow diamond	self.consultations: list[Consultation]
ClinicManager -> BillingRecord	Hollow diamond	self.billing_records: list[BillingRecord]



Test-Case Table

#	Test Objective	Input / Precondition	Expected Result	Actual Result	Status
1	Register three patient subclasses	Option 1: P97, Test1, 111, 20, 1 Option 1: P98, Test2, 222, 65, 2, SC123 Option 1: P99, Test3, 333, 30, 3, 5	System creates Regular, Senior, and Emergency objects correctly.	Displays "Patient registered successfully." three times.	Pass
2	Reject a duplicate patient ID	Option 1: P01, Duplicate, 000, 25, 1 (Precondition: P01 is already hardcoded)	System rejects registration and raises custom exception.	Displays [Business Rule Error] Duplicate error: Patient ID already exists.	Pass



3	Reject a duplicate queue entry	Option 3: P01 <i>(Precondition: P01 is already in the hardcoded active queue)</i>	System rejects duplicate queue addition.	Displays [Business Rule Error] Duplicate error: Patient is already in the active queue.	Pass
4	Verify priority then queue-number ordering	Option 3: P03 Then Option 8 to view queue. <i>(Precondition: P01 Regular and P02 Senior are already in queue)</i>	Queue automatically sorts: P03 (Emergency) becomes first, P02 (Senior) second, P01 (Regular) third.	Report shows Waiting Patients order as: 1. Bob Rush, 2. Jane Smith, 3. John Doe.	Pass
5	Complete a valid consultation	Option 5: P03, D01, Checkup, 1000, Done	Consultation completes, updates doctor count, and bill B001 is generated.	Displays Consultation complete. Bill 'B001'	Pass



		<i>(Precondition: P03 must be first in queue after Test 4)</i>		generated for PHP 1000.00	
6	Reject completion without a doctor	Option 5: P02, D99, Checkup, 500, Notes <i>(Precondition: P02 is now first in queue; D99 does not exist)</i>	System aborts transaction safely without changing state.	Displays [Business Rule Error] Validation error: Doctor ID not found.	Pass
7	Prevent double completion	Option 5: P01, D02, Therapy, 800, Notes <i>(Precondition: P02 is currently first in queue, not P01)</i>	System prevents processing a patient out of order.	Displays [Business Rule Error] Validation error: This patient is not next in the queue.	Pass



8	Compute bills for all patient types	Option 5: P02, D02, X-Ray, 1000, Done Then Option 7: B002	P02 (Senior) receives a computed bill with a 20% discount.	Bill B002 receipt shows Subtotal: PHP 1000.00, Discount: PHP 200.00, Final: PHP 800.00.	Pass
9	Recover from invalid input conversion	Option 1: P10, Test Name, 0999, abc (Expects integer for Age)	System catches ValueError without crashing and returns to the menu.	Displays [Input Error] Invalid data format. Please check your inputs.	Pass
10	Register domain class (Doctor)	Option 2: D04, Dr. Strange, Surgery	System successfully creates and stores the Doctor object.	Displays Doctor registered successfully.	Pass



11	Reject duplicate doctor ID	Option 2: D01, Dr. Fake, FakeSpec (Precondition: D01 is already hardcoded)	System catches the error and rejects registration to protect state integrity.	Displays [Business Rule Error] Duplicate error: Doctor ID already exists.	Pass
12	Search for an existing patient	Option 6: P01 (Precondition: P01 is already hardcoded)	System accesses the dictionary and polymorphically prints the object's __str__.	Displays Found: [P01] John Doe (Age: 30) - Contact: 555-0101	Pass
13	Reject search for missing record	Option 6: P999 (Precondition: P999 has not been registered)	System gracefully handles the missing key and raises the custom exception.	Displays [Business Rule Error] Patient not found in records.	Pass



14	Reject invalid billing record print	Option 7: B999 <i>(Precondition: No consultation has generated bill B999 yet)</i>	System searches the transaction list, fails to find it, and raises an exception safely.	Displays [Business Rule Error] Billing record not found.	Pass
15	Protect state on invalid consultation charge	Option 5: P01, D01, Checkup, free (Expects float for base charge) <i>(Precondition: P01 is next in queue)</i>	System catches ValueError before instantiating the Consultation object, leaving the queue and doctor count completely unchanged.	Displays [Input Error] Invalid data format. Please check your inputs. (Queue remains intact).	Pass
16	Reject calling next patient from an empty queue	Option 4 <i>(Precondition: The active queue has 0 patients)</i>	System catches the empty list safely and raises a custom exception instead of crashing with an IndexError.	Displays [Business Rule Error] The queue is currently empty.	Pass

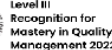


17	Verify highest priority patient is called first	Option 4 (Precondition: Queue contains P01 Regular, P02 Senior, and P03 Emergency)	System polymorphically checks priorities and retrieves the Emergency patient first.	Displays Next patient in queue: [P03] Bob Rush (Age: 45)	Pass
18	Verify calling patient does not dequeue them	Option 4 followed immediately by Option 4 again (Precondition: P03 is currently first in the queue)	System displays the same patient both times, proving that "calling" only views the next patient and leaves them in the queue until a consultation is officially completed.	Displays Next patient in queue: [P03] Bob Rush (Age: 45) twice consecutively.	Pass
19	Exit the system	Option 9	System terminates the application and displays an exit message.	Displays Exiting system. Goodbye!	Pass



TEST-CASES SCREENSHOTS

#	Actual Result
1	<pre>===== CLINIC QUEUE CONSULTATION AND BILLING SYSTEM ===== [1] Register patient [2] Register doctor [3] Add patient to queue [4] Call next patient [5] Complete consultation [6] Search patient [7] Print bill [8] Generate reports [9] Exit Select an option (1-9): 1 >>> REGISTER PATIENT Patient ID: P99 Name: Test3 Contact: 333 Age: 30 Type (1: Regular, 2: Senior, 3: Emergency): 3 Severity Level (1-10): 5 [SUCCESS] Patient registered successfully. ===== CLINIC QUEUE CONSULTATION AND BILLING SYSTEM ===== [1] Register patient [2] Register doctor [3] Add patient to queue [4] Call next patient [5] Complete consultation [6] Search patient [7] Print bill [8] Generate reports [9] Exit Select an option (1-9): 1 >>> REGISTER PATIENT Patient ID: P99 Name: Test3 Contact: 333 Age: 30 Type (1: Regular, 2: Senior, 3: Emergency): 3 Severity Level (1-10): 5 [SUCCESS] Patient registered successfully.</pre> <pre>===== CLINIC QUEUE CONSULTATION AND BILLING SYSTEM ===== [1] Register patient [2] Register doctor [3] Add patient to queue [4] Call next patient [5] Complete consultation [6] Search patient [7] Print bill [8] Generate reports [9] Exit Select an option (1-9): 1 >>> REGISTER PATIENT Patient ID: P97 Name: Test1 Contact: 111 Age: 20 Type (1: Regular, 2: Senior, 3: Emergency): 1 [SUCCESS] Patient registered successfully.</pre>



2

```
=====
CLINIC QUEUE CONSULTATION AND BILLING SYSTEM
=====

[1] Register patient
[2] Register doctor
[3] Add patient to queue
[4] Call next patient
[5] Complete consultation
[6] Search patient
[7] Print bill
[8] Generate reports
[9] Exit

-----
Select an option (1-9): 1
-----

>>> REGISTER PATIENT
Patient ID: P01
Name: Duplicate
Contact: 000
Age: 25
Type (1: Regular, 2: Senior, 3: Emergency): 1

[ERROR - Business Rule] Patient ID already exists.
```

3

```
=====
CLINIC QUEUE CONSULTATION AND BILLING SYSTEM
=====

[1] Register patient
[2] Register doctor
[3] Add patient to queue
[4] Call next patient
[5] Complete consultation
[6] Search patient
[7] Print bill
[8] Generate reports
[9] Exit

-----
Select an option (1-9): 3
-----

>>> ADD PATIENT TO QUEUE
Enter Patient ID to add to queue: P01

[ERROR - Business Rule] Patient is already in the active queue.
```



4	<pre>----- Select an option (1-9): 3 ----- >>> ADD PATIENT TO QUEUE Enter Patient ID to add to queue: P03 [SUCCESS] Patient added to queue successfully.</pre>
5	<pre>----- Select an option (1-9): 5 ----- >>> COMPLETE CONSULTATION Confirm Patient ID: P03 Enter Doctor ID: D01 Service rendered: Checkup Base charge amount: 1000 Completion details/notes: Done [SUCCESS] Consultation complete. Bill 'B001' generated for PHP 1000.00</pre>
6	<pre>----- Select an option (1-9): 5 ----- >>> COMPLETE CONSULTATION Confirm Patient ID: P02 Enter Doctor ID: D99 Service rendered: Checkup Base charge amount: 500 Completion details/notes: Notes [ERROR - Business Rule] Doctor ID not found.</pre>



7

```
-----
Select an option (1-9): 5
-----

>>> COMPLETE CONSULTATION
Confirm Patient ID: P01
Enter Doctor ID: D02
Service rendered: Therapy
Base charge amount: 800
Completion details/notes: Notes

[ERROR - Business Rule] This patient is not next in the queue.
```

8

```
-----
Select an option (1-9): 5
-----

>>> COMPLETE CONSULTATION
Confirm Patient ID: P02
Enter Doctor ID: D02
Service rendered: X-ray
Base charge amount: 1000
Completion details/notes: Done

[SUCCESS] Consultation complete.
Bill 'B002' generated for PHP 800.00
```

```
-----
Select an option (1-9): 7
-----

>>> PRINT BILL
Enter Bill ID to print: B002

--- OFFICIAL RECEIPT ---
Bill ID:      B002
Subtotal:    PHP 1,000.00
Discount:    PHP 200.00
Final Amount: PHP 800.00
-----
```



9

```
-----
Select an option (1-9): 1
-----

>>> REGISTER PATIENT
Patient ID: P10
Name: Test Name
Contact: 0999
Age: abc
      [!] Invalid input. Please enter a whole number.
Age: 40
Type (1: Regular, 2: Senior, 3: Emergency): 1

[SUCCESS] Patient registered successfully.
```

10

```
-----
Select an option (1-9): 2
-----

>>> REGISTER DOCTOR
Doctor ID: D04
Name: Dr. Strange
Specialization: Surgery

[SUCCESS] Doctor registered successfully.
```



11	<pre>----- Select an option (1-9): 2 ----- >>> REGISTER DOCTOR Doctor ID: D01 Name: Dr. Fake Specialization: FakeSpec [ERROR - Business Rule] Doctor ID already exists.</pre>
12	<pre>----- Select an option (1-9): 6 ----- >>> SEARCH PATIENT Enter Patient ID to search: P01 [RESULT] [P01] Aldous David (Age: 30) Contact Info: 555-0101</pre>
13	<pre>----- Select an option (1-9): 6 ----- >>> SEARCH PATIENT Enter Patient ID to search: P999 [ERROR - Business Rule] Patient not found in records.</pre>



14	<pre>----- Select an option (1-9): 7 ----- >>> PRINT BILL Enter Bill ID to print: 8999 [ERROR - Business Rule] Billing record not found.</pre>
15	<pre>----- Select an option (1-9): 5 ----- >>> COMPLETE CONSULTATION Confirm Patient ID: P01 Enter Doctor ID: D01 Service rendered: Checkup Base charge amount: fre [!] Invalid input. Please enter a valid decimal number. Base charge amount: 1000 Completion details/notes: Notes [ERROR - Business Rule] This patient is not next in the queue.</pre>
16	<pre>===== CLINIC QUEUE CONSULTATION AND BILLING SYSTEM ===== [1] Register patient [2] Register doctor [3] Add patient to queue [4] Call next patient [5] Complete consultation [6] Search patient [7] Print bill [8] Generate reports [9] Exit ----- Select an option (1-9): 4 ----- >>> CALL NEXT PATIENT [ERROR - Business Rule] The queue is currently empty.</pre>



17

```
=====
CLINIC QUEUE CONSULTATION AND BILLING SYSTEM
=====

[1] Register patient
[2] Register doctor
[3] Add patient to queue
[4] Call next patient
[5] Complete consultation
[6] Search patient
[7] Print bill
[8] Generate reports
[9] Exit

-----

Select an option (1-9): 4

-----

>>> CALL NEXT PATIENT
Next patient in queue:
-> [P02] Lancelot Lizano (Age: 68)
```

18

```
=====
[1] Register patient
[2] Register doctor
[3] Add patient to queue
[4] Call next patient
[5] Complete consultation
[6] Search patient
[7] Print bill
[8] Generate reports
[9] Exit

-----

Select an option (1-9): 4

-----

>>> CALL NEXT PATIENT
Next patient in queue:
-> [P02] Lancelot Lizano (Age: 68)
```

```
=====
CLINIC QUEUE CONSULTATION AND BILLING SYSTEM
=====

[1] Register patient
[2] Register doctor
[3] Add patient to queue
[4] Call next patient
[5] Complete consultation
[6] Search patient
[7] Print bill
[8] Generate reports
[9] Exit

-----

Select an option (1-9): 4

-----

>>> CALL NEXT PATIENT
Next patient in queue:
-> [P02] Lancelot Lizano (Age: 68)
```



19

```
=====
CLINIC QUEUE CONSULTATION AND BILLING SYSTEM
=====
[1] Register patient
[2] Register doctor
[3] Add patient to queue
[4] Call next patient
[5] Complete consultation
[6] Search patient
[7] Print bill
[8] Generate reports
[9] Exit
-----
Select an option (1-9): 9
-----
Exiting system. Goodbye!

Process finished with exit code 0
```